

UNIVERSIDAD DE GUAYAQUIL
FACULTAD DE CIENCIAS MATEMÁTICAS Y FÍSICAS
CARRERA DE SOFTWARE

Manual Técnico

Sistema de Chat en Tiempo Real

Integrantes:

César Tipán Antón
Erick Cordova
Camillie Ayovi
Mateo Aguilar

Docente:

Ing. Joel Alejandro Barba Salazar, Mg.

Asignatura:

Aplicaciones Distribuidas

Guayaquil, julio de 2026

Índice general

Manual técnico	1
1. Descripción general	1
2. Arquitectura del sistema	1
3. Tecnologías utilizadas y dependencias	2
4. Estructura del proyecto	4
5. Instalación y configuración	4
6. Ejecución local	6
7. Funcionamiento del servidor	6
8. Funcionamiento del cliente	8
9. Comunicación mediante sockets	9
10. Modelo de datos	10
11. Pruebas	10
12. Despliegue	10
13. Limitaciones conocidas	13

Manual técnico

Sistema de chat en tiempo real sobre WebSockets. Documento dirigido a desarrolladores y evaluadores técnicos.

1. Descripción general

ChatApp es una aplicación de mensajería instantánea con arquitectura cliente–servidor. Varios clientes se conectan de forma simultánea a un servidor centralizado, se autentican con credenciales propias y se comunican en tiempo real dentro de canales temáticos. El servidor mantiene el estado de presencia de cada canal y difunde los mensajes a todos los participantes conectados.

El sistema se compone de tres piezas desplegadas por separado:

Pieza	Responsabilidad
Cliente web (Angular)	Interfaz de usuario, gestión de sesión y consumo del canal WebSocket
Servidor (FastAPI)	Autenticación, API REST, punto WebSocket, difusión de mensajes y registro de presencia
Base de datos (PostgreSQL)	Persistencia de usuarios, canales e historial de mensajes

2. Arquitectura del sistema

La comunicación se reparte en dos canales con propósitos distintos:

- **HTTPS/REST** para operaciones puntuales sin estado: registro, inicio de sesión y consulta del catálogo de canales. Cada petición viaja con su propio token y el servidor no guarda sesión.
- **WSS (WebSocket sobre TLS)** para el chat propiamente dicho. Es una conexión persistente y full-duplex: el servidor empuja mensajes y cambios de presencia al cliente sin que este los solicite.

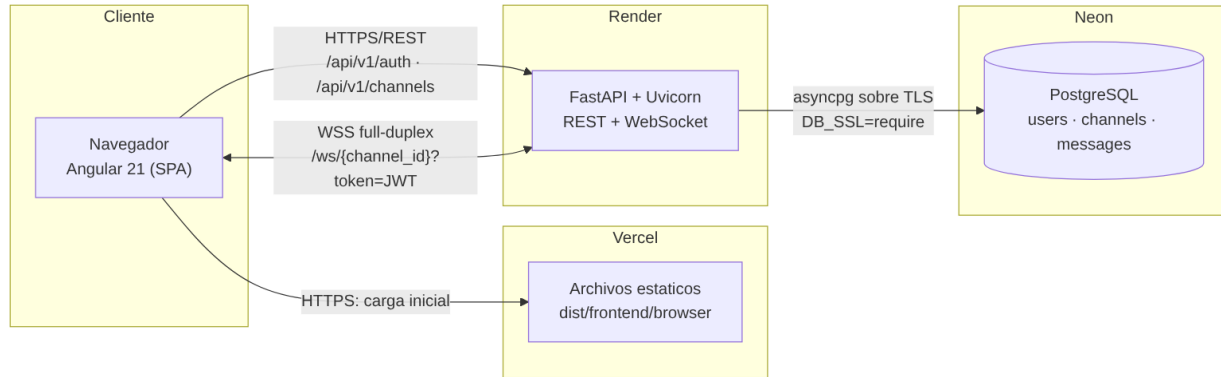


Figura 1: Arquitectura de despliegue

2.1 Justificación del uso de WebSockets

El enunciado admite «sockets o WebSockets». Se optó por WebSockets por tres razones:

1. Un socket TCP crudo no puede exponerse públicamente en las plataformas gratuitas de despliegue (Render, Railway, Vercel), que solo publican los puertos HTTP/HTTPS. El entregable de «enlace público desplegado» sería inviable.
2. WebSocket es una API de sockets sobre TCP: conserva el modelo de conexión persistente y bidireccional que exige el tema del proyecto, y añade el handshake HTTP necesario para atravesar proxies y balanceadores.
3. Permite reutilizar el mismo origen y el mismo certificado TLS que la API REST, lo que simplifica la configuración de CORS y de seguridad de transporte.

Sobre HTTP convencional habría sido necesario recurrir a *polling*, que introduce latencia y carga innecesaria; el servidor no puede iniciar la comunicación.

2.2 Organización interna del servidor

El backend sigue una separación por capas. Cada capa solo conoce la inmediatamente inferior:

- **Presentación** (api/v1/, sockets/handler.py): traduce HTTP y WebSocket a llamadas de negocio. No contiene reglas de dominio.
- **Negocio** (services/, sockets/manager.py): reglas de la aplicación. ConnectionManager mantiene en memoria el estado volátil de las conexiones.
- **Acceso a datos** (repositories/): única capa que emite consultas SQL, mediante SQLAlchemy en modo asíncrono.
- **Transversales** (core/): configuración, hashing de contraseñas y emisión/verificación de tokens.

El estado de presencia vive en memoria y no se persiste: es información efímera, válida solo mientras la conexión existe. El historial de mensajes, en cambio, se escribe en PostgreSQL.

3. Tecnologías utilizadas y dependencias

3.1 Servidor

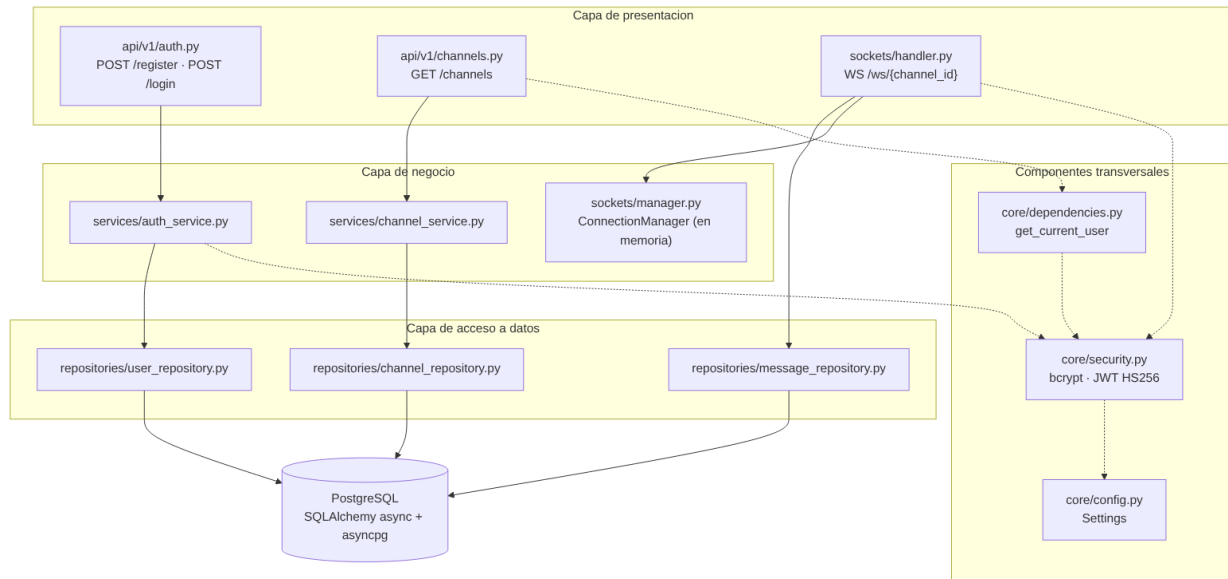


Figura 2: Componentes del backend

Dependencia	Versión	Función
FastAPI	0.111.0	Framework HTTP y WebSocket
Uvicorn	0.30.1	Servidor ASGI
SQLAlchemy	2.0.31	ORM asíncrono
asyncpg	0.29.0	Controlador de PostgreSQL
python-jose	3.3.0	Firma y verificación de JWT
bcrypt	4.0.1	Hashing de contraseñas
pydantic-settings	2.3.4	Carga tipada de configuración
pytest / pytest-asyncio	7.4.4 / 0.23.3	Pruebas unitarias

Tiempo de ejecución: Python 3.12 (imagen python:3.12-slim).

3.2 Cliente

Dependencia	Versión	Función
Angular	21.1	Framework de la interfaz
RxJS	7.8	Flujos asíncronos sobre HTTP
TypeScript	5.9	Lenguaje
Vitest	4.0	Pruebas unitarias
Playwright	1.61	Pruebas de extremo a extremo

El cliente usa componentes *standalone*, señales (*signals*) para el estado reactivo y el flujo de control de plantillas `@if` / `@for`.

3.3 Infraestructura

Servicio	Uso
Vercel	Publicación de los archivos estáticos del cliente
Render	Ejecución del contenedor del servidor
Neon	PostgreSQL gestionado, con TLS obligatorio

4. Estructura del proyecto

ug-disc-chat/	
├─ backend/	
│ └─ app/	
│ └─ main.py	Punto de entrada: CORS y montaje de routers
│ └─ api/v1/	Rutas REST (auth, channels)
│ └─ core/	Configuración, seguridad y dependencias
│ └─ db/	Motor, sesión y modelos SQLAlchemy
│ └─ repositories/	Consultas SQL
│ └─ schemas/	Contratos Pydantic de entrada y salida
│ └─ services/	Reglas de negocio
│ └─ sockets/	ConnectionManager y punto WebSocket
│ └─ tests/	Pruebas unitarias
│ └─ Dockerfile	
│ └─ requirements.txt	
├─ frontend/	
│ └─ src/app/	
│ └─ core/	Servicios, guard, interceptor y modelos
│ └─ features/auth/	Inicio de sesión y registro
│ └─ features/chat/	Disposición, canales, mensajes y presencia
│ └─ shared/ui/	Logotipo e iconos SVG
│ └─ e2e/	Pruebas de extremo a extremo
│ └─ src/environments/	Direcciones del servidor por entorno
├─ docs/	Manuales, diagramas y plan de pruebas
└─ schema.sql	Esquema y datos iniciales de la base

5. Instalación y configuración

5.1 Requisitos previos

- Python 3.12 o superior
- Node.js 20 o superior
- Una instancia de PostgreSQL 14 o superior, local o gestionada

5.2 Base de datos

Ejecutar `schema.sql` contra la base de datos. El script crea las tablas `users`, `channels` y `messages`, el índice de historial e inserta los tres canales iniciales (`general`, `tech`, `off-topic`). Es idempotente: puede ejecutarse más de una vez sin duplicar los canales.

El proyecto no incorpora una herramienta de migraciones. Todo cambio de esquema se aplica ejecutando el script a mano.

5.3 Servidor

```
cd backend
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install -r requirements.txt
cp .env.example .env              # Completar los valores
```

Variables de entorno:

Variable	Obligatoria	Valor por defecto	Descripción
JWT_SECRET_KEY	Sí	—	Clave de firma de los tokens. Se recomienda una clave robusta de al menos 32 caracteres
DATABASE_URL	Sí	—	Cadena de conexión con el formato post-gresql+asyncpg://usuario:clave@host:puerto
DB_SSL	No	require	Modo TLS de asyncpg. Los PostgreSQL gestionados exigen require; una instancia local no negocia TLS y necesita disable
ALLOWED_ORIGINS	No	["http://localhost:4200"]	Lista de orígenes autorizados por CORS
JWT_ALGORITHM	No	HS256	Algoritmo de firma
JWT_EXPIRE_MINUTES	No	1440	Vigencia del token, en minutos
DEBUG	No	False	Registra en consola las sentencias SQL

El archivo .env no se versiona.

5.4 Cliente

```
cd frontend
npm install
```

Las direcciones del servidor no se leen de variables de entorno, sino de dos archivos que el compilador intercambia según la configuración:

Archivo	Configuración	apiUrl	wsUrl
environment.ts	desarrollo	http://localhost:8000/api/	ws://localhost:8000/ws
environment.prod.ts	producción	https://ug-disc-chat.onrender.com/api/	wss://ug-disc-chat.onrender.com/ws

6. Ejecución local

Se necesitan dos terminales. El servidor debe arrancar primero.

Terminal 1

```
cd backend && source .venv/bin/activate
uvicorn app.main:app --reload
```

Terminal 2

```
cd frontend && npm start
```

- Cliente: `http://localhost:4200`
- API: `http://localhost:8000`
- Documentación interactiva de la API: `http://localhost:8000/docs`
- Sonda de estado: `http://localhost:8000/health`

Con una base de datos local, `DB_SSL` debe valer `disable`; en caso contrario el servidor arranca y responde en `/health`, pero toda consulta falla al intentar negociar TLS.

7. Funcionamiento del servidor

7.1 API REST

Método	Ruta	Cuerpo	Respuesta	Código	Token
POST	<code>/api/v1/auth/register</code>	<code>{name, email, password}</code>	<code>access_token,</code> <code>token_type,</code> <code>user_id,</code> <code>username</code>	201	No
POST	<code>/api/v1/auth/login</code>	<code>{email, password}</code>	Igual que el anterior	200	No
GET	<code>/api/v1/channels/</code>	—	Lista de canales	200	Sí
GET	<code>/health</code>	—	<code>status, app</code>	200	No

Errores relevantes: 409 si el correo o el nombre de usuario ya están registrados; 401 ante credenciales incorrectas o token inválido o expirado.

7.2 Autenticación

Las contraseñas nunca se almacenan ni se transmiten en claro hacia la base de datos. `bcrypt.hashpw` genera un *salt* aleatorio por usuario, de modo que dos contraseñas idénticas producen hashes distintos y quedan protegidas frente a tablas precalculadas. La verificación usa `bcrypt.checkpw`, que compara en tiempo constante.

El servidor emite un JWT firmado con HS256 que contiene el identificador del usuario (`sub`), su nombre y la marca de expiración (`exp`), con una vigencia de 24 horas. El servidor no guarda sesiones: cada petición se valida con la firma del token.

Ante credenciales incorrectas la respuesta es siempre la misma (401, «Credenciales incorrectas»), sin distinguir si el fallo fue el correo o la contraseña. La respuesta genérica evita revelar de forma explícita qué correos

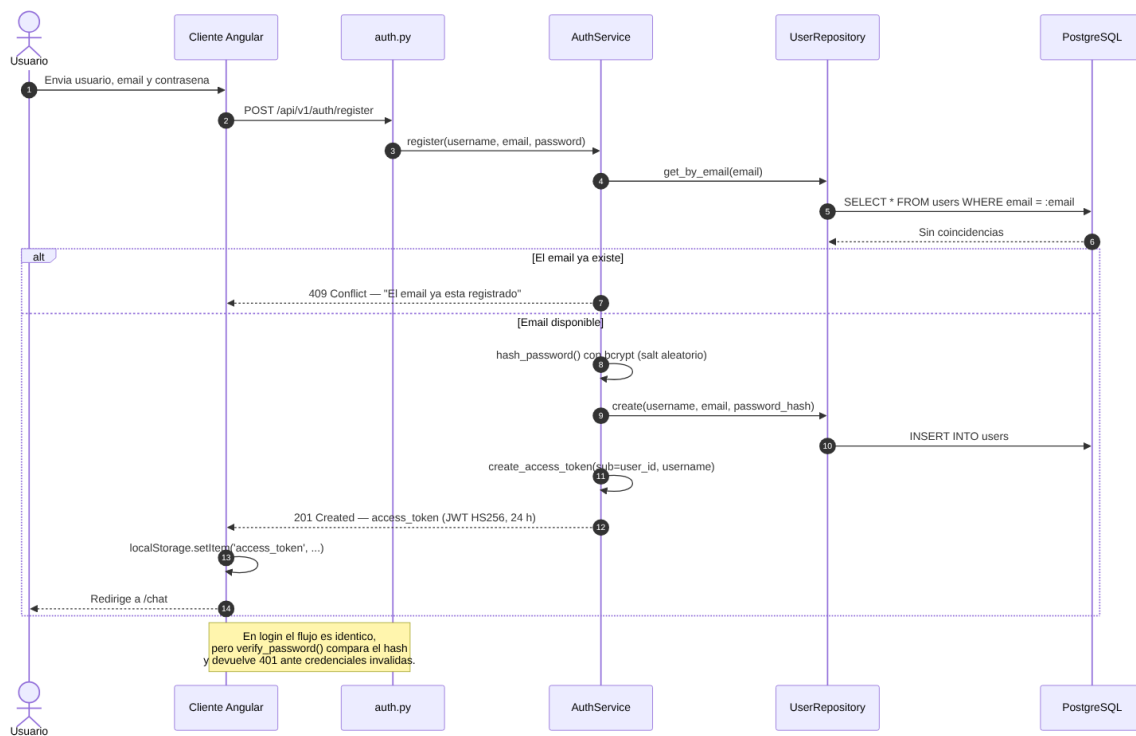


Figura 3: Secuencia de autenticación

existen en el sistema, aunque una diferencia de tiempo en la ejecución de `bcrypt` podría permitir enumerar cuentas mediante análisis fino.

7.3 Integridad y seguridad de los datos transmitidos

- **Transporte cifrado.** En producción todo el tráfico viaja por TLS: HTTPS para REST y WSS para el WebSocket. La conexión con la base de datos también se cifra (`DB_SSL=require`).
- **Control de origen.** El middleware CORS solo admite los orígenes declarados en `ALLOWED_ORIGINS`, y limita métodos y cabeceras. No se usa el comodín `*`.
- **Validación de entrada.** Los esquemas Pydantic rechazan las peticiones mal formadas antes de que lleguen a la lógica de negocio: el correo debe tener formato válido, el nombre de usuario entre 3 y 50 caracteres, y la contraseña un mínimo de 8. El cliente limita los mensajes a 2000 caracteres.
- **Prevención de inyección SQL.** Las consultas se construyen con expresiones parametrizadas de SQLAlchemy; nunca por concatenación de cadenas.
- **Autorización en el WebSocket.** El token se valida antes de aceptar la conexión. Sin token válido no se completa el handshake.

7.4 Gestión de conexiones simultáneas

`ConnectionManager` (`app/sockets/manager.py`) mantiene el registro de quién está conectado y a qué canal:

```
_rooms[channel_id][user_id] = { "username": str, "sockets": set[WebSocket] }
```

Cada usuario tiene un **conjunto** de sockets, no uno solo. Esto permite que la misma cuenta esté abierta en

varias pestañas o dispositivos: el usuario aparece una sola vez en la lista de conectados y solo se le da por desconectado cuando se cierra su última conexión.

El servidor es asíncrono y de un solo hilo (bucle de eventos de asyncio), por lo que las operaciones sobre estas estructuras no requieren bloqueos. Antes de difundir, el gestor materializa la lista de destinatarios, porque el propio envío puede modificar el diccionario al detectar sockets muertos; si un envío falla, el socket se descarta en el acto y la difusión continúa con el resto.

8. Funcionamiento del cliente

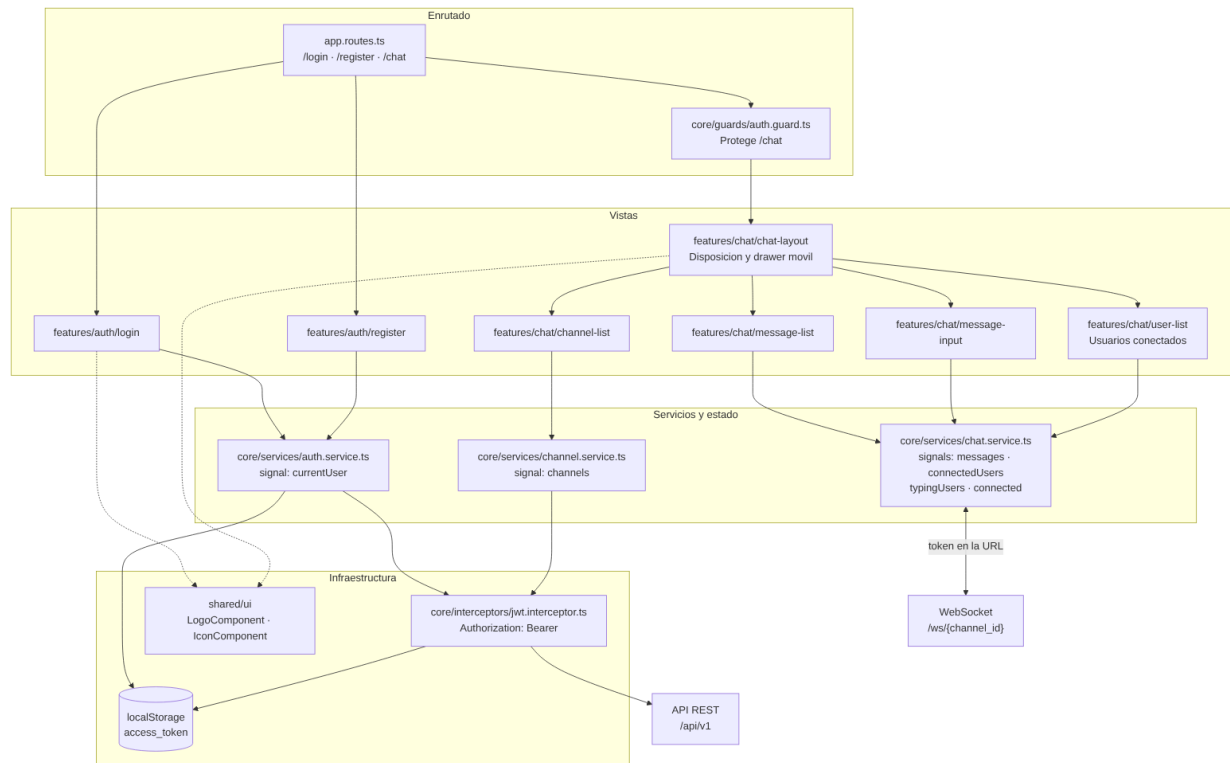


Figura 4: Componentes del cliente

Aplicación de página única con tres rutas: /login, /register y /chat. Esta última está protegida por un *guard* que redirige a /login cuando no hay sesión.

Tras un inicio de sesión correcto, el token y los datos del usuario se guardan en `localStorage`. Un interceptor los adjunta como cabecera `Authorization: Bearer <token>` en toda petición HTTP saliente. El WebSocket no puede llevar cabeceras propias, de modo que el token viaja como parámetro de consulta en la URL de conexión.

El estado de la conversación vive en `ChatService` y se expone mediante señales que las plantillas consumen directamente: `messages`, `connectedUsers`, `typingUsers` y `connected`. Cuando llega un evento por el socket, el servicio actualiza la señal correspondiente y Angular vuelve a dibujar solo lo que cambió.

Al seleccionar otro canal, el cliente cierra la conexión anterior y abre una nueva contra el canal elegido, limpiando mensajes y lista de conectados.

9. Comunicación mediante sockets

9.1 Establecimiento de la conexión

Punto de conexión: `wss://<host>/ws/{channel_id}?token=<JWT>`

El servidor valida el token y el identificador del canal **antes** de aceptar el handshake. Si algo falla, cierra la conexión con un código propio:

Código	Motivo
4001	Token ausente, inválido o expirado
4002	El identificador de canal no es un UUID válido
4004	Canal inexistente (UUID válido pero el canal no existe en la BD)

Una vez aceptada la conexión, el servidor envía al recién llegado los últimos 20 mensajes del canal (`history_batch`), anuncia su entrada a los demás (`user_joined`) y difunde la lista completa de conectados a todo el canal (`user_list`).

9.2 Ciclo de vida de la conexión

9.3 Protocolo de mensajes

Todos los eventos son objetos JSON con un campo discriminador `type`.

Del cliente al servidor

type	Carga útil	Efecto
message	content	Persiste el mensaje y lo difunde al canal. Si el contenido está vacío tras recortar espacios, se descarta
typing	is_typing	Se difunde a los demás. No se persiste

Del servidor al cliente

type	Carga útil	Cuándo se emite
history_batch	messages[]	Una sola vez, al conectar, y solo al socket entrante
message	id, channel_id, user_id, username, content, created_at	Ante cada mensaje válido; a todo el canal, incluido el emisor
typing	user_id, username, is_typing	A todo el canal excepto el emisor
user_list	users[] con user_id y username	Al conectar y al desconectar un usuario; a todo el canal

type	Carga útil	Cuándo se emite
user_joined	user_id, username	Solo en la primera conexión del usuario; excluye al que entra
user_left	user_id, username	Solo cuando el usuario cierra su última conexión

user_list es la única fuente de verdad de la lista de conectados. user_joined y user_left son notificaciones puntuales; el cliente no reconstruye la lista a partir de ellas, porque un cliente que se conecta tarde no habría visto las entradas anteriores.

9.4 Envío y recepción de mensajes

El servidor difunde el mensaje a todos los miembros del canal, incluido quien lo escribió. El emisor no dibuja el mensaje al pulsar Enter, sino cuando lo recibe de vuelta: así el mensaje que se muestra es exactamente el que quedó persistido, con su identificador y su marca de tiempo definitivos.

9.5 Presencia y conexiones múltiples

La rutina de limpieza se ejecuta dentro de un bloque finally. Si el manejador termina por una excepción inesperada, el usuario se retira igualmente de la lista de conectados; de lo contrario quedarían usuarios fantasma, visibles como conectados pero sin socket detrás.

10. Modelo de datos

Las claves primarias son UUID, no enteros secuenciales: no revelan cuántos usuarios o mensajes existen ni permiten enumerarlos.

Las dos claves foráneas de messages se comportan de forma distinta a propósito. Al borrar un canal, sus mensajes se eliminan en cascada, porque fuera del canal carecen de sentido. Al borrar un usuario, sus mensajes se conservan con user_id a nulo y se muestran como «Usuario eliminado»: así no se abren huecos en la conversación de los demás.

El índice (channel_id, created_at DESC) sostiene la única consulta caliente del sistema: recuperar los últimos mensajes de un canal al conectarse.

11. Pruebas

El proyecto tiene 84 pruebas automatizadas: 46 unitarias del servidor, 29 unitarias del cliente y 9 de extremo a extremo sobre dos navegadores reales. El alcance, la estrategia y los resultados se detallan en plan-de-pruebas.md.

```
cd backend && python -m pytest tests -q      # 46
cd frontend && npm test                        # 29
cd frontend && npx playwright test            # 9, requiere servidor y base de datos
```

12. Despliegue

El despliegue es continuo: cada integración en la rama main publica una nueva versión.

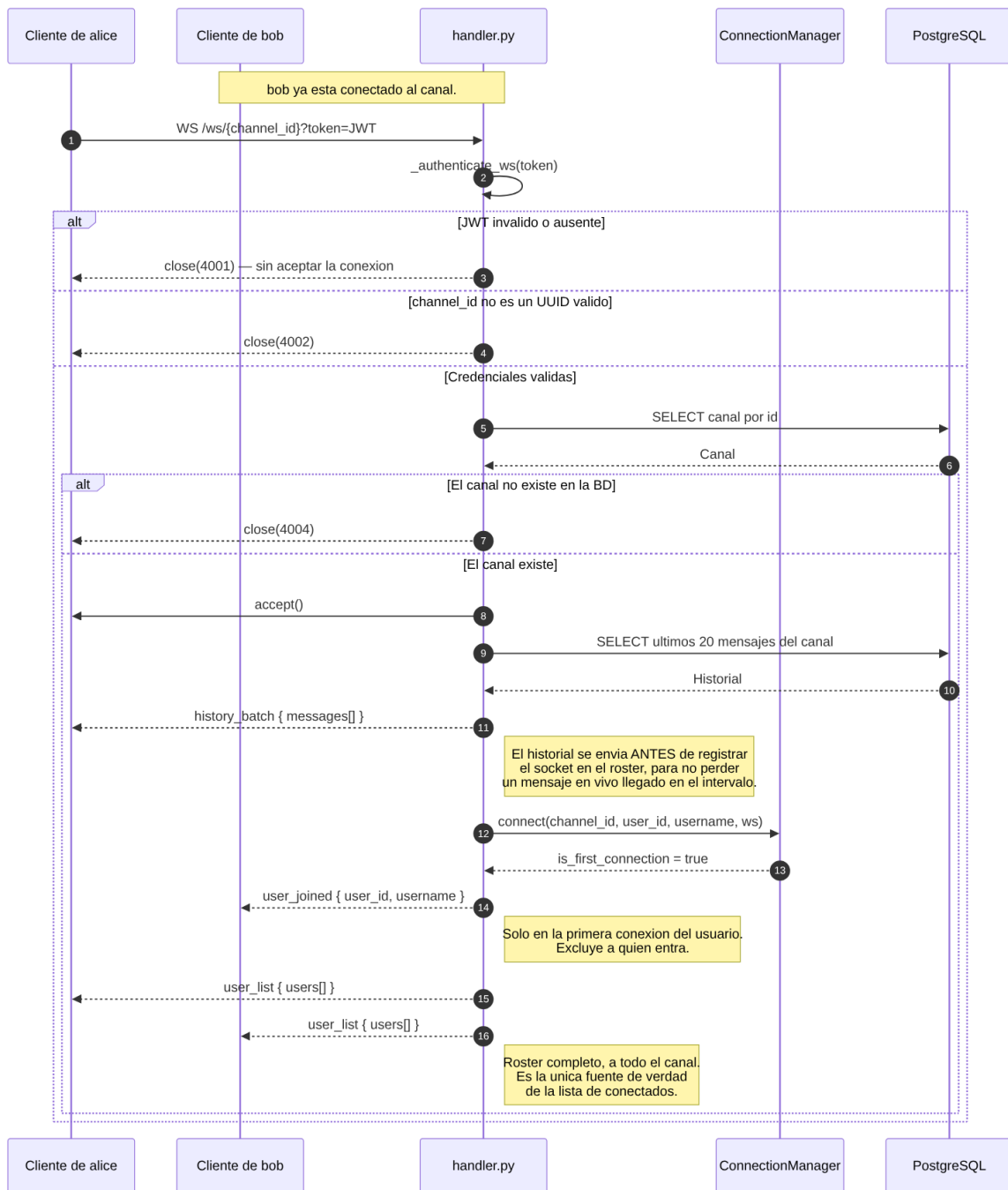


Figura 5: Secuencia de conexión

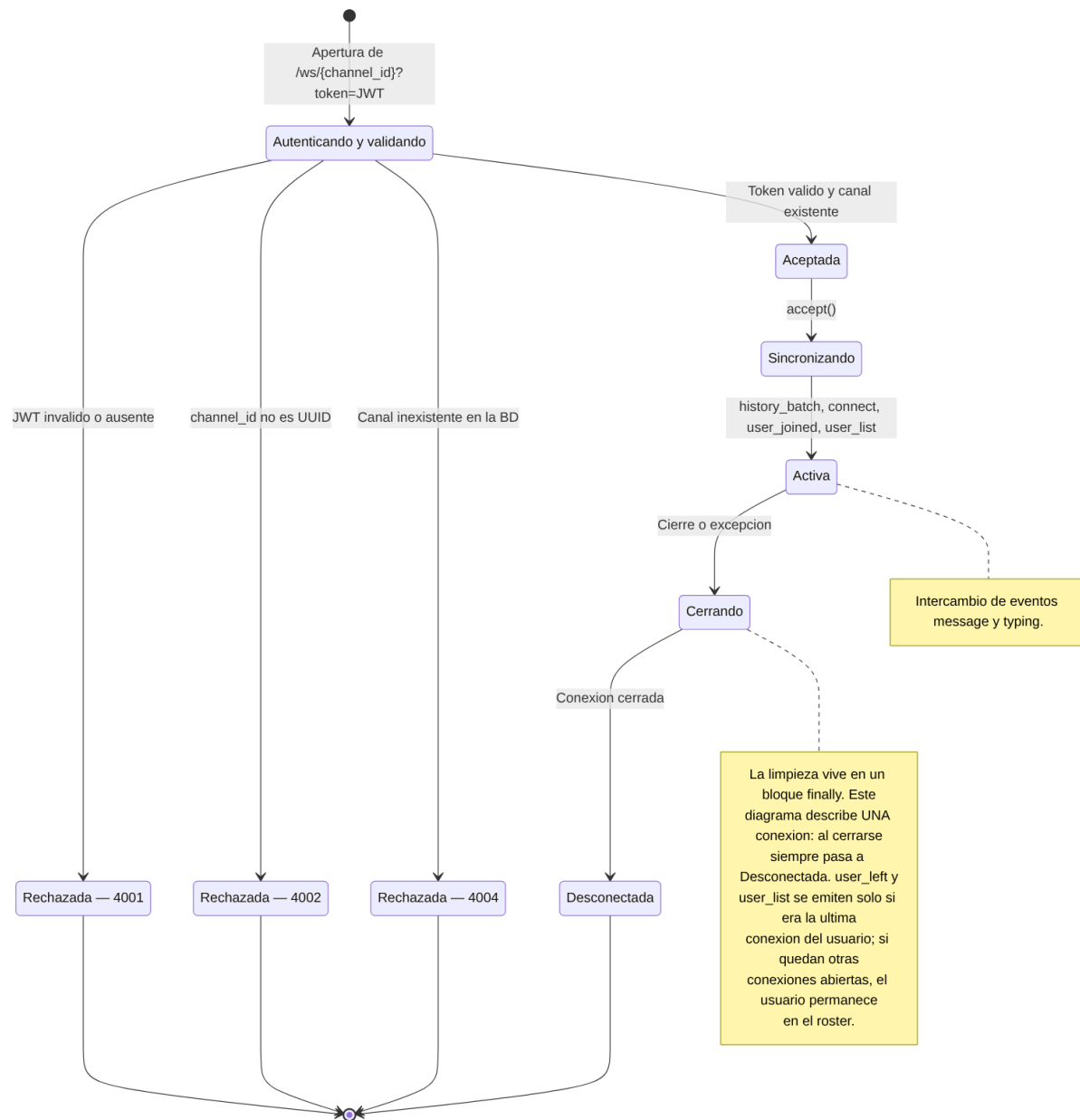


Figura 6: Estados de la conexión

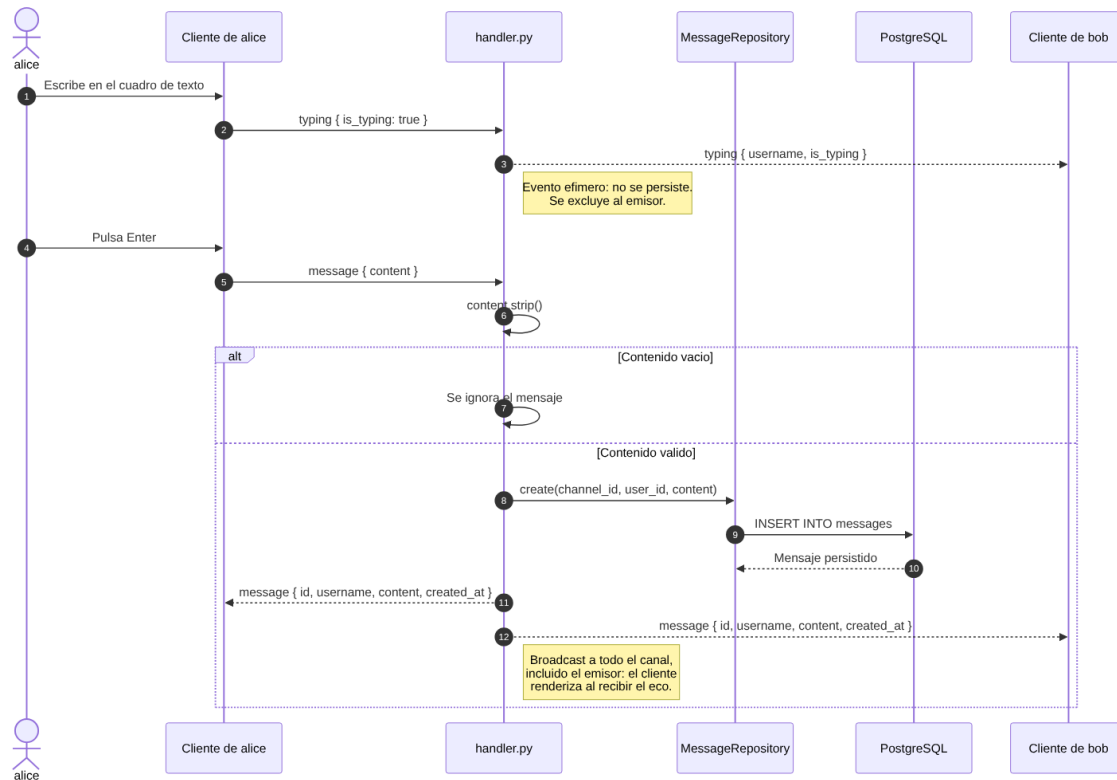


Figura 7: Secuencia de envío de mensaje

Servidor (Render). Construye la imagen a partir de backend/Dockerfile, en dos etapas para no arrastrar los compiladores a la imagen final. Arranca con `uvicorn app.main:app --host 0.0.0.0 --port $PORT` y se comprueba con la sonda `/health`. Las variables de entorno se cargan desde el panel del servicio, nunca desde el repositorio.

Cliente (Vercel). Compila con `npm run build -- --configuration production` y publica `dist/frontend/browser`. Al ser una aplicación de página única, todas las rutas se reescriben hacia `index.html`.

Base de datos (Neon). PostgreSQL gestionado con TLS obligatorio.

Al publicar el cliente hay que verificar que `environment.prod.ts` apunte a la dirección vigente del servidor, ya que ese valor se fija en tiempo de compilación.

13. Limitaciones conocidas

- **No hay reconexión automática.** Si la conexión WebSocket se corta, el cliente marca el estado como desconectado pero no reintenta. El usuario debe volver a seleccionar el canal.
- **La expiración del token no se gestiona en el cliente.** Transcurridas 24 horas las peticiones empiezan a fallar con 401 sin redirigir a la pantalla de inicio de sesión.
- **La presencia es por proceso.** ConnectionManager guarda el estado en la memoria del servidor. Con más de una instancia, cada una vería solo a sus propios conectados. Escalar horizontalmente exigiría un canal de publicación y suscripción compartido, por ejemplo Redis.
- **El plan gratuito de Render suspende el servicio por inactividad.** La primera petición tras un periodo

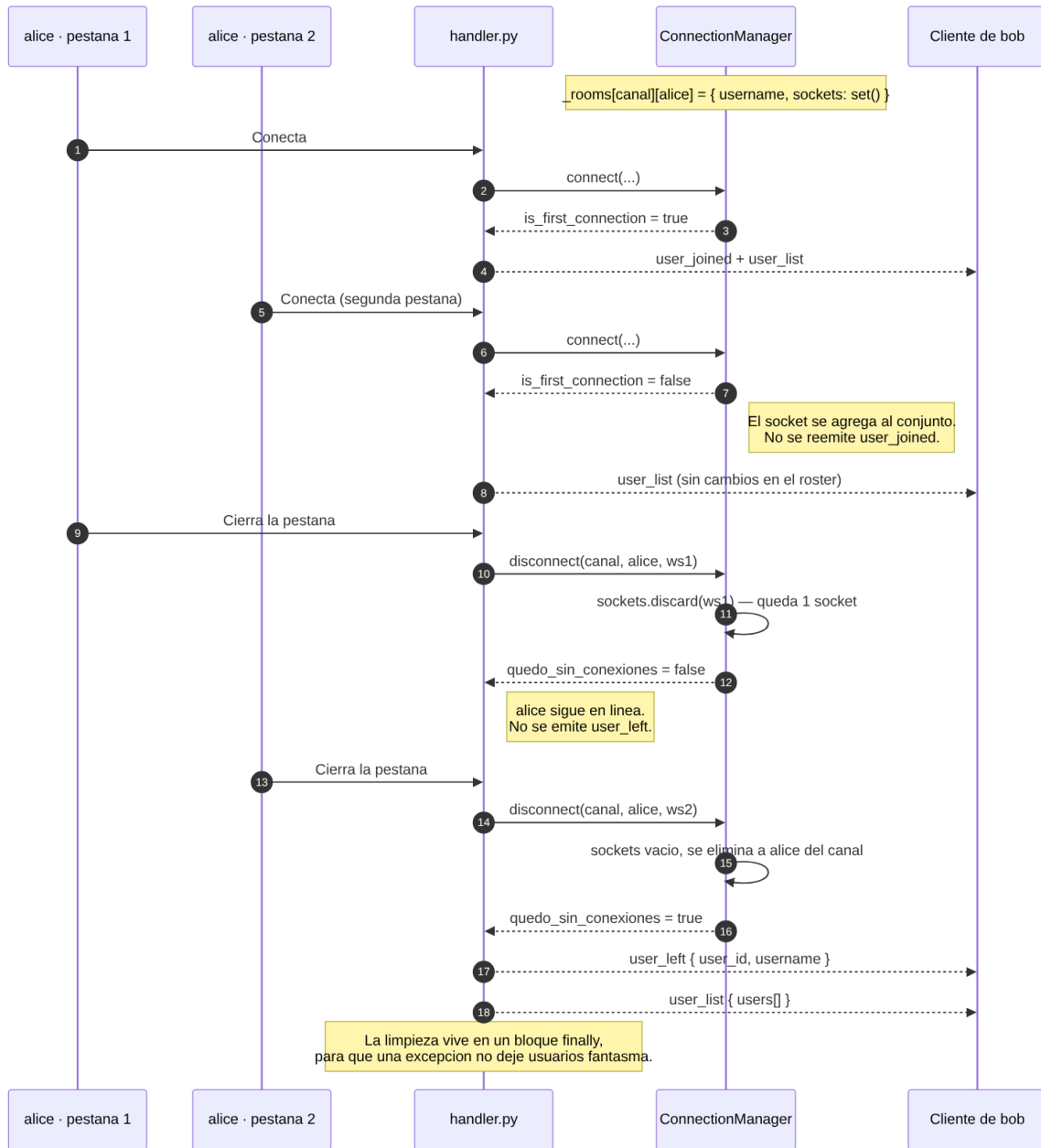


Figura 8: Secuencia de presencia con múltiples conexiones

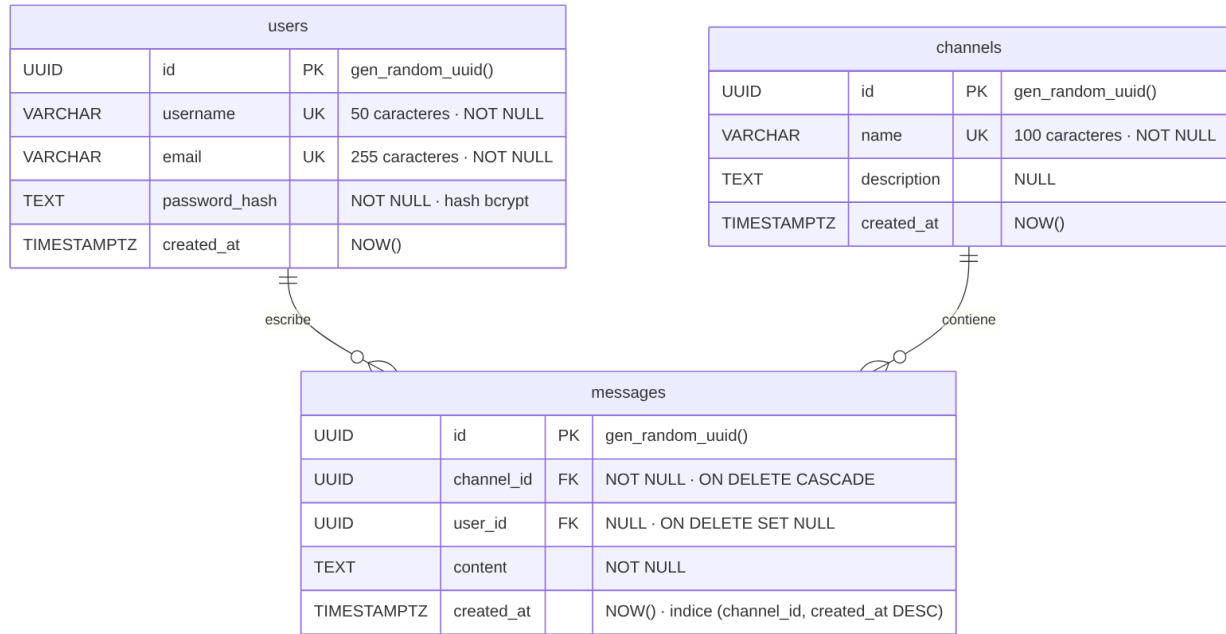


Figura 9: Modelo entidad-relación

de reposo puede tardar cerca de un minuto.

- **Los canales son fijos.** Se crean desde el script de esquema; no existe un punto de creación de canales en la API.